Bank marketing dataset — data processing

End-to-end loading, quality checks, cleaning, and feature preparation for `data/bank-full.csv` (UCI Bank Marketing).

1. Exploratory Data Analysis

In [62]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split

sns.set_theme(style="whitegrid", palette="deep")
pd.set_option("display.max_columns", 25)

DATA_PATH = Path("data/bank-full.csv")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

1.1 Load raw data

In [25]:
```
assert DATA_PATH.exists(), f"Missing file: {DATA_PATH.resolve()}"

df = pd.read_csv(DATA_PATH, sep=";")
print("Shape:", df.shape)
df.head()
```

```
Shape: (45211, 17)
```

Out[25]:

	age	job	marital	education	default	balance	housing	loan	contact	d
0	58	management	married	tertiary	no	2143	yes	no	unknown	
1	44	technician	single	secondary	no	29	yes	no	unknown	
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	
4	33	unknown	single	unknown	no	1	no	no	unknown	

1.2. Schema, types, and missing values

In [26]:
```
display(df.info())

na_counts = df.isna().sum()
na_counts[na_counts > 0]
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   age        45211 non-null  int64 
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64 
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64 
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64 
 12  campaign   45211 non-null  int64 
 13  pdays      45211 non-null  int64 
 14  previous   45211 non-null  int64 
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
None
```

Out[26]:
```
Series([], dtype: int64)
```

In [27]:
```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = [c for c in df.columns if c not in numeric_cols and c !=

print("Numeric:", numeric_cols)
print("Categorical:", categorical_cols)

df.describe().T
```

```
Numeric: ['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previou
s']
Categorical: ['job', 'marital', 'education', 'default', 'housing', 'loan',
'contact', 'month', 'poutcome']
```

Out[27]:

	count	mean	std	min	25%	50%	75%	max
age	45211.0	40.936210	10.618762	18.0	33.0	39.0	48.0	95.0
balance	45211.0	1362.272058	3044.765829	-8019.0	72.0	448.0	1428.0	102127.0
day	45211.0	15.806419	8.322476	1.0	8.0	16.0	21.0	31.0
duration	45211.0	258.163080	257.527812	0.0	103.0	180.0	319.0	4918.0
campaign	45211.0	2.763841	3.098021	1.0	1.0	2.0	3.0	63.0
pdays	45211.0	40.197828	100.128746	-1.0	-1.0	-1.0	-1.0	871.0
previous	45211.0	0.580323	2.303441	0.0	0.0	0.0	0.0	275.0

1.3 Sentinel values and implicit missingness

In this dataset, `pdays == -1` means the client was not contacted in the previous campaign (not a real day count). We record that explicitly below.

In [28]:
```
unknown_like = df[categorical_cols].eq("unknown").sum().sort_values(ascendir
unknown_like[unknown_like > 0]
```

Out[28]:
```
poutcome     36959
contact      13020
education     1857
job            288
dtype: int64
```

1.4 Duplicates and consistency

In [29]:
```
dup_mask = df.duplicated()
n_dup = dup_mask.sum()
print(f"Duplicate rows (all columns): {n_dup}")

key_cols = [
    c for c in df.columns if c not in {"duration", "campaign"}
]  # exclude call outcomes if modeling leakage
dup_key = df.duplicated(subset=key_cols)
print(f"Duplicates on subset (excluding duration, campaign): {dup_key.sum()}

df_clean = df.loc[~dup_mask].copy()
print("After dropping exact duplicates:", df_clean.shape)
```

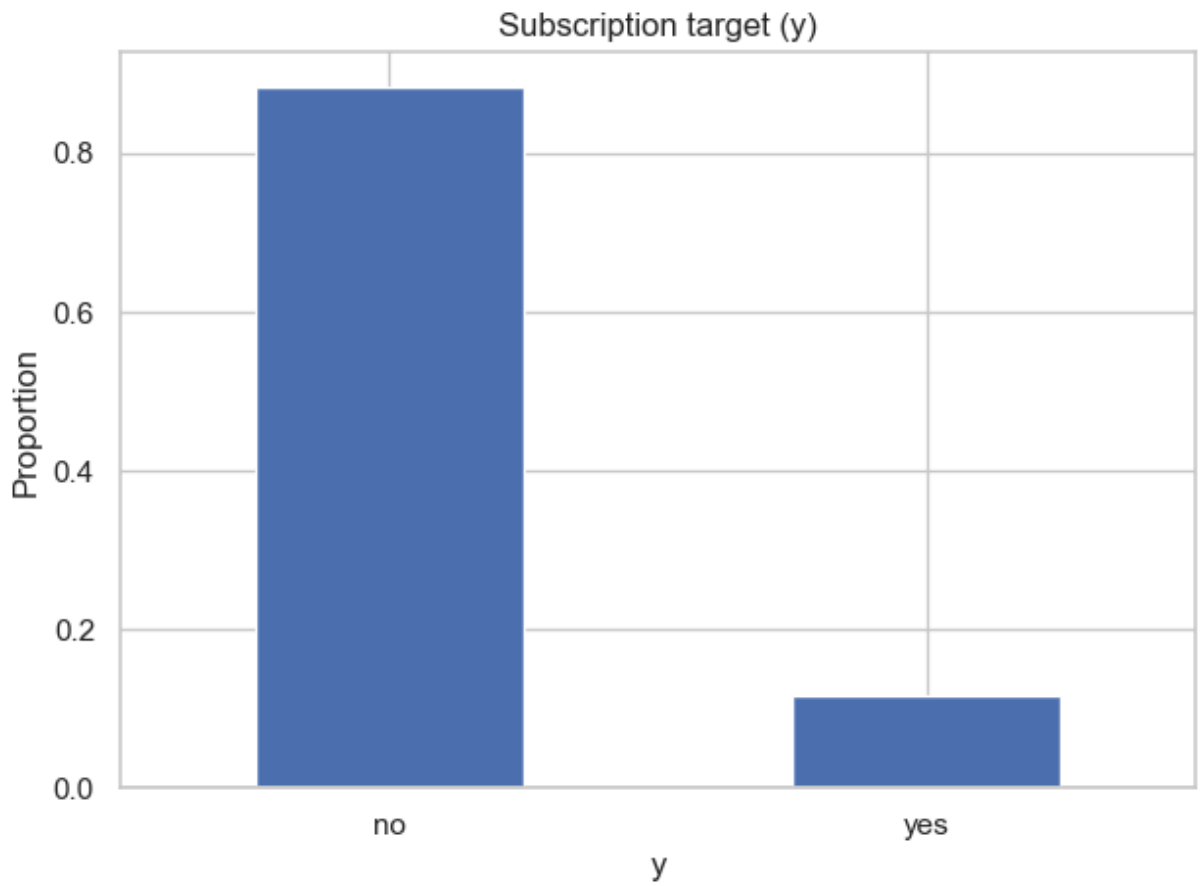
```
Duplicate rows (all columns): 0
Duplicates on subset (excluding duration, campaign): 53
After dropping exact duplicates: (45211, 17)
```

1.5 Target variable and class balance

In [30]:
```
target = "y"
vc = df_clean[target].value_counts(normalize=True)
display(vc)
```

```
vc.plot(kind="bar", title="Subscription target (y)", rot=0)
plt.ylabel("Proportion")
plt.tight_layout()
plt.show()
```

```
y
no     0.883015
yes    0.116985
Name: proportion, dtype: float64
```

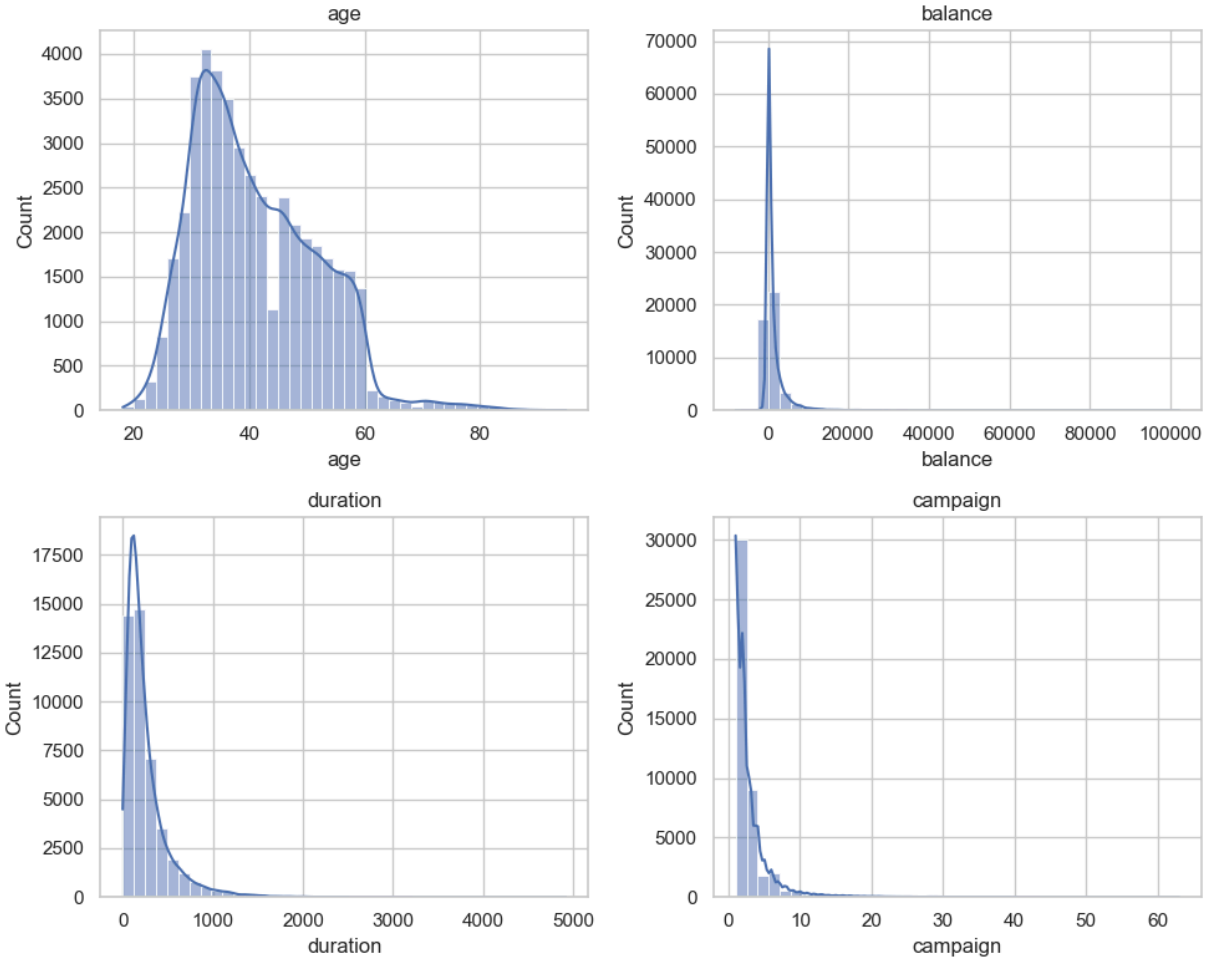


1.6. Outliers and distributions (numeric)

In [31]:
```
def iqr_bounds(s: pd.Series, k: float = 1.5) -> tuple[float, float]:
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr


for col in ["balance", "duration", "campaign", "age"]:
    lo, hi = iqr_bounds(df_clean[col])
    out = ((df_clean[col] < lo) | (df_clean[col] > hi)).sum()
    print(f"{col}: IQR outliers = {out} (bounds [{lo:.2f}, {hi:.2f}])")

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, col in zip(axes.ravel(), ["age", "balance", "duration", "campaign"]):
    sns.histplot(df_clean[col], bins=40, kde=True, ax=ax)
    ax.set_title(col)
```

```
plt.tight_layout()
plt.show()
```

```
balance: IQR outliers = 4729 (bounds [-1962.00, 3462.00])
duration: IQR outliers = 3235 (bounds [-221.00, 643.00])
campaign: IQR outliers = 3064 (bounds [-2.00, 6.00])
age: IQR outliers = 487 (bounds [10.50, 70.50])
```



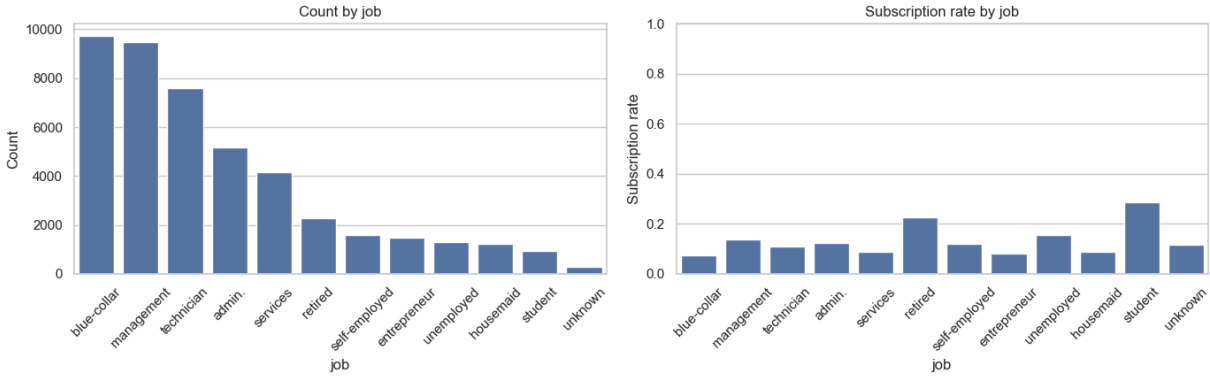
1.7. Target-wise EDA for categorical variables

1.7.1. job

In [32]:
```
col = "job"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
```

```
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	blue-collar	9732	0.215257	0.072750
1	management	9458	0.209197	0.137556
2	technician	7597	0.168034	0.110570
3	admin.	5171	0.114375	0.122027
4	services	4154	0.091880	0.088830
5	retired	2264	0.050076	0.227915
6	self-employed	1579	0.034925	0.118429
7	entrepreneur	1487	0.032890	0.082717
8	unemployed	1303	0.028820	0.155027
9	housemaid	1240	0.027427	0.087903
10	student	938	0.020747	0.286780
11	unknown	288	0.006370	0.118056

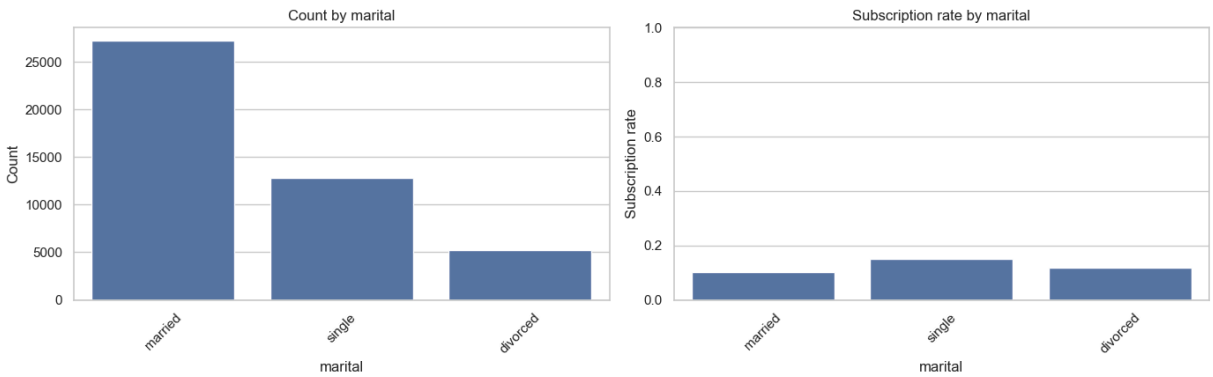


1.7.2. marital

In [33]:
```
col = "marital"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)
```

```
plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	married	27214	0.601933	0.101235
1	single	12790	0.282896	0.149492
2	divorced	5207	0.115171	0.119455

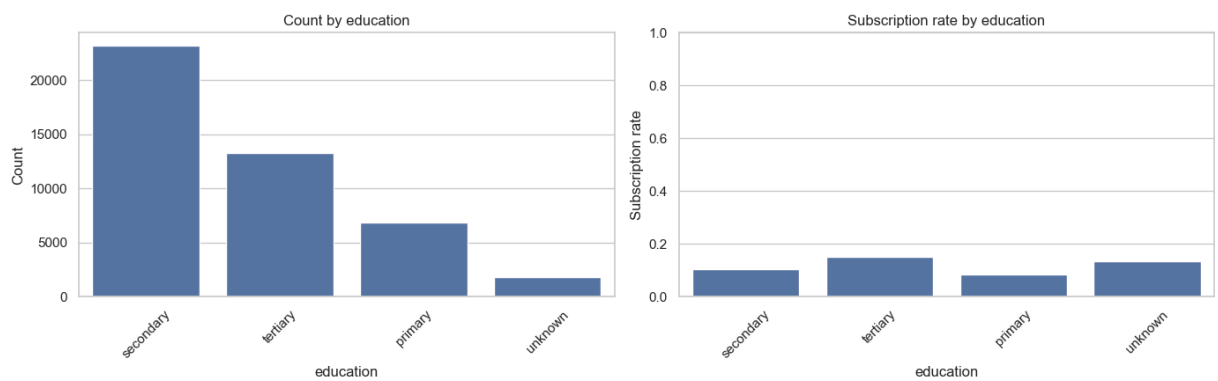


1.7.3. education

In [34]:
```
col = "education"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
```

```
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	secondary	23202	0.513194	0.105594
1	tertiary	13301	0.294198	0.150064
2	primary	6851	0.151534	0.086265
3	unknown	1857	0.041074	0.135703

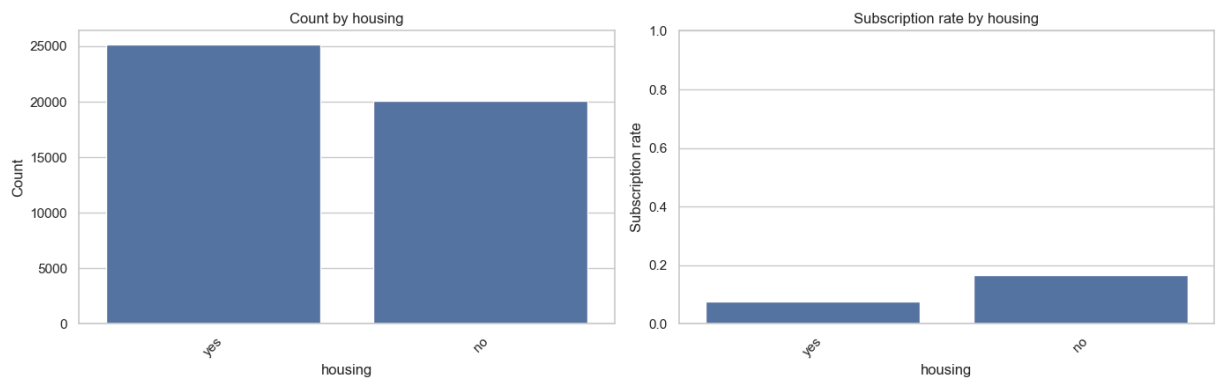


1.7.3. housing

In [35]:
```
col = "housing"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
```

```
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	yes	25130	0.555838	0.077000
1	no	20081	0.444162	0.167024

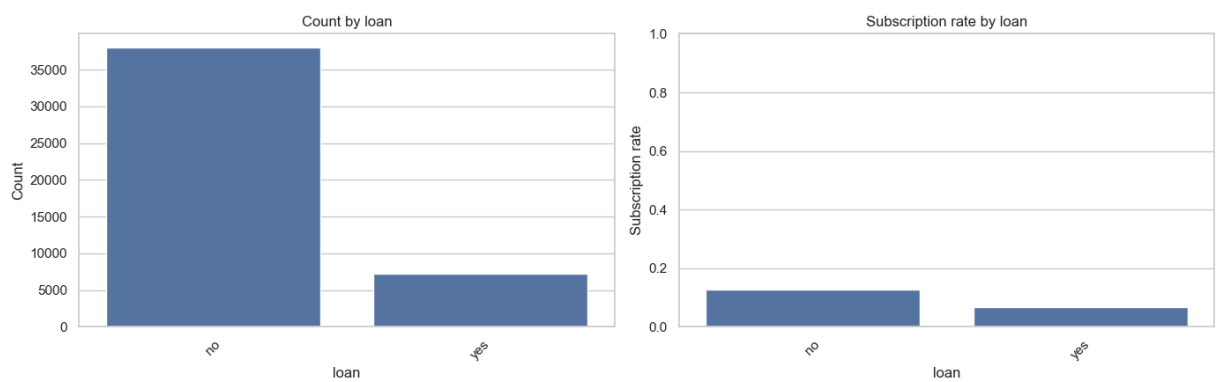


1.7.5. loan

In [36]:
```
col = "loan"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
```

```
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	no	37967	0.839774	0.126557
1	yes	7244	0.160226	0.066814

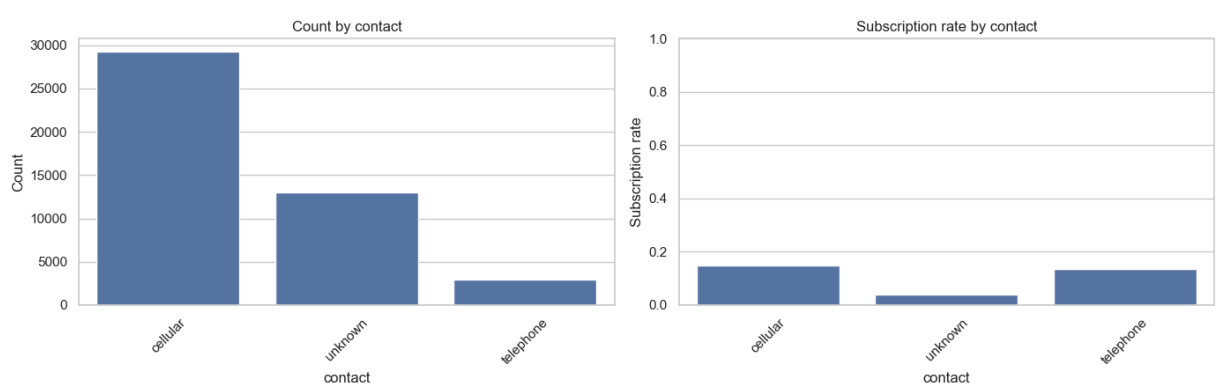


1.7.6. contact

In [37]:
```
col = "contact"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
```

```
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

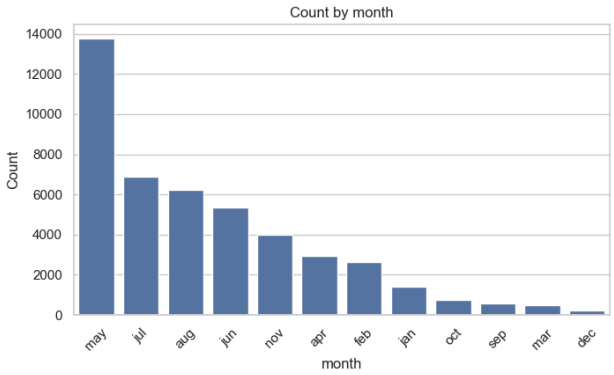
	category	count	proportion	subscription_rate
0	cellular	29285	0.647741	0.149189
1	unknown	13020	0.287983	0.040707
2	telephone	2906	0.064276	0.134205

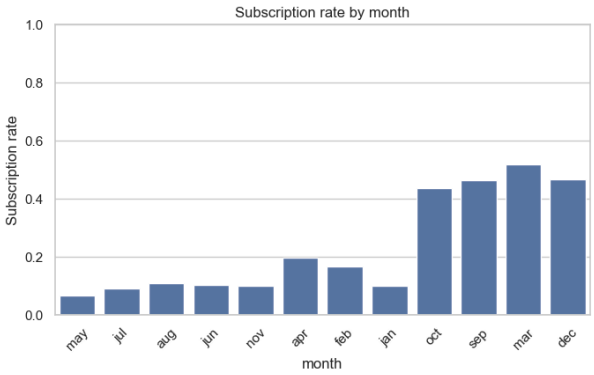


1.7.7. month

In [38]:
```
col = "month"

summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	may	13766	0.304483	0.067195
1	jul	6895	0.152507	0.090935
2	aug	6247	0.138174	0.110133
3	jun	5341	0.118135	0.102228
4	nov	3970	0.087810	0.101511
5	apr	2932	0.064851	0.196794
6	feb	2649	0.058592	0.166478
7	jan	1403	0.031032	0.101212
8	oct	738	0.016323	0.437669
9	sep	579	0.012807	0.464594
10	mar	477	0.010551	0.519916
11	dec	214	0.004733	0.467290



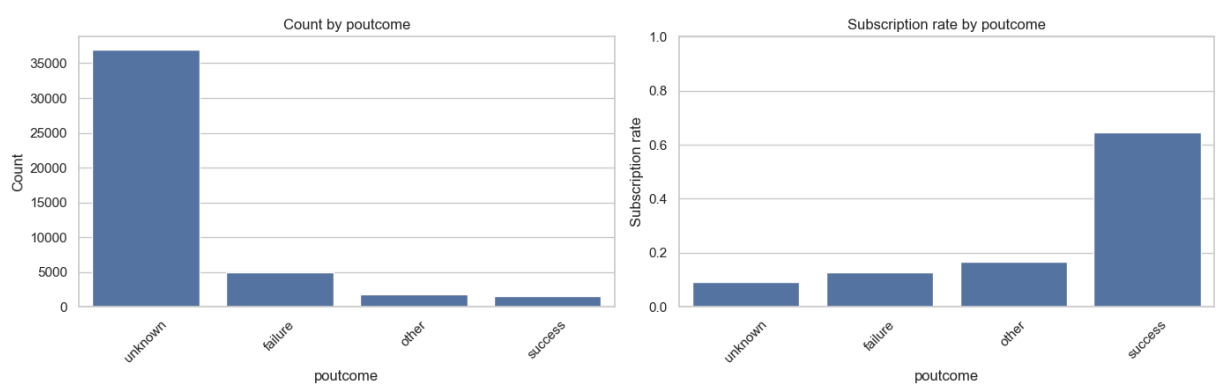


1.7.8. poutcome

In [39]:
```
col = "poutcome"
summary = (
    df_clean.groupby(col, dropna=False)
    .agg(count=("y", "size"))
    .reset_index()
    .rename(columns={col: "category"})
    .sort_values("count", ascending=False)
    .reset_index(drop=True)
)

subscription_rate = (
    df_clean.groupby(col, dropna=False)["y"]
    .apply(lambda s: (s == "yes").mean())
    .reset_index(name="subscription_rate")
    .rename(columns={col: "category"})
)

summary = summary.merge(subscription_rate, on="category", how="left")
```

```
summary["proportion"] = summary["count"] / len(df_clean)
summary = summary[["category", "count", "proportion", "subscription_rate"]]

display(summary)

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

sns.countplot(data=df_clean, x=col, order=summary["category"], ax=axes[0])
axes[0].set_title(f"Count by {col}")
axes[0].set_xlabel(col)
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=45)

sns.barplot(data=summary, x="category", y="subscription_rate", ax=axes[1])
axes[1].set_title(f"Subscription rate by {col}")
axes[1].set_xlabel(col)
axes[1].set_ylabel("Subscription rate")
axes[1].set_ylim(0, 1)
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

	category	count	proportion	subscription_rate
0	unknown	36959	0.817478	0.091615
1	failure	4901	0.108403	0.126097
2	other	1840	0.040698	0.166848
3	success	1511	0.033421	0.647253



1.8 Correlation among numeric / engineered columns

2.Feature engineering

2.1 Create core engineered features

Create the main cleaned and engineered feature set.

In [ ]:

```
processed = df_clean.copy()

processed["y_binary"] = (processed["y"] == "yes").astype(np.int8)

processed["pdays_was_contacted"] = (processed["pdays"] != -1).astype(np.int8
processed["pdays_since_prev"] = processed["pdays"].where(
    processed["pdays"] != -1, np.nan
)

month_order = [
    "jan",
    "feb",
    "mar",
    "apr",
    "may",
    "jun",
    "jul",
    "aug",
    "sep",
    "oct",
    "nov",
    "dec",
]
processed["month"] = pd.Categorical(
    processed["month"], categories=month_order, ordered=True
)
processed["month_ord"] = (
    processed["month"].cat.codes.replace(-1, np.nan).astype("Int64")
)

education_order = ["primary", "secondary", "tertiary", "unknown"]
processed["education"] = pd.Categorical(
    processed["education"], categories=education_order, ordered=True
)
processed["education_ord"] = (
    processed["education"].cat.codes.replace(-1, np.nan).astype("Int64")
)

for c in ["default", "housing", "loan"]:
    processed[f"{c}_binary"] = (processed[c] == "yes").astype(np.int8)

processed["balance_signed_log"] = np.sign(processed["balance"]) * np.log1p(
    np.abs(processed["balance"])
)

display(
    processed[
        [
            "y_binary",
            "pdays_was_contacted",
            "pdays_since_prev",
            "month_ord",
            "education_ord",
            "default_binary",
            "housing_binary",
```

```
            "loan_binary",
            "balance_signed_log",
        ]
    ].head()
)
```

	y_binary	pdays_was_contacted	pdays_since_prev	month_ord	education_ord	defau
0	0	0	NaN	4	2	
1	0	0	NaN	4	1	
2	0	0	NaN	4	1	
3	0	0	NaN	4	3	
4	0	0	NaN	4	3	

2.2 Add non-overlapping derived behavioral features

Add a small set of additional behavioral features with low overlap with the core engineered features.

In [ ]:
```
new_columns = []

processed["multiple_contacts_current_campaign"] = (processed["campaign"] > 1
    np.int8
)
new_columns.append("multiple_contacts_current_campaign")

processed["poutcome_success"] = (processed["poutcome"] == "success").astype(
new_columns.append("poutcome_success")

processed["poutcome_failure"] = (processed["poutcome"] == "failure").astype(
new_columns.append("poutcome_failure")

processed["contact_is_unknown"] = (processed["contact"] == "unknown").astype
new_columns.append("contact_is_unknown")

processed["poutcome_is_unknown"] = (processed["poutcome"] == "unknown").asty
new_columns.append("poutcome_is_unknown")

print("Newly added columns:", new_columns)
display(processed[new_columns].head())
```

```
Newly added columns: ['multiple_contacts_current_campaign', 'poutcome_succes
s', 'poutcome_failure', 'contact_is_unknown', 'poutcome_is_unknown']
```

	multiple_contacts_current_campaign	poutcome_success	poutcome_failure	contact_i
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

2.3. Add a small set of interaction features

Create compact interaction indicators from existing contact-history and loan-related feature columns.

In [42]:
```
new_columns = []

processed["contacted_before_and_success"] = (
    processed["pdays_was_contacted"] * processed["poutcome_success"]
).astype(np.int8)
new_columns.append("contacted_before_and_success")

processed["contacted_before_and_failure"] = (
    processed["pdays_was_contacted"] * processed["poutcome_failure"]
).astype(np.int8)
new_columns.append("contacted_before_and_failure")

processed["many_contacts_and_housing_loan"] = (
    processed["multiple_contacts_current_campaign"] * processed["housing_bir
).astype(np.int8)
new_columns.append("many_contacts_and_housing_loan")

processed["previous_and_many_current_contacts"] = (
    processed["pdays_was_contacted"] * processed["multiple_contacts_current_
).astype(np.int8)
new_columns.append("previous_and_many_current_contacts")

processed["both_housing_and_personal_loan"] = (
    processed["housing_binary"] * processed["loan_binary"]
).astype(np.int8)
new_columns.append("both_housing_and_personal_loan")

processed["history_info_missing"] = (
    processed["contact_is_unknown"] * processed["poutcome_is_unknown"]
).astype(np.int8)
new_columns.append("history_info_missing")

print("Newly added columns:", new_columns)
display(processed[new_columns].head())
```

```
Newly added columns: ['contacted_before_and_success', 'contacted_before_and_
failure', 'many_contacts_and_housing_loan', 'previous_and_many_current_conta
cts', 'both_housing_and_personal_loan', 'history_info_missing']
```

	contacted_before_and_success	contacted_before_and_failure	many_contacts_and_hc
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

2.4. Add binned versions of selected numeric features

Bin `age`, `balance`, `campaign`, and `pdays_since_prev` into interpretable ordinal categories for downstream modelling.

In [43]:
```
new_columns = []

# age → age_group
processed["age_group"] = pd.cut(
    processed["age"],
    bins=[17, 25, 35, 50, np.inf],
    labels=["18–25", "26–35", "36–50", "51+"],
).astype(str)
new_columns.append("age_group")

# balance → balance_bucket
processed["balance_bucket"] = pd.cut(
    processed["balance"],
    bins=[-np.inf, 0, 500, 2000, np.inf],
    labels=["negative", "low", "medium", "high"],
    right=False,
).astype(str)
new_columns.append("balance_bucket")

# campaign → campaign_bucket
processed["campaign_bucket"] = pd.cut(
    processed["campaign"],
    bins=[0, 1, 3, np.inf],
    labels=["1", "2–3", "4+"],
).astype(str)
new_columns.append("campaign_bucket")

# pdays_since_prev → pdays_recency_bucket
processed["pdays_recency_bucket"] = np.select(
    [
        processed["pdays_was_contacted"] == 0,
        processed["pdays_since_prev"] <= 30,
        processed["pdays_since_prev"] <= 180,
    ],
    ["no_previous_contact", "recent", "medium"],
    default="old",
)
```

```
new_columns.append("pdays_recency_bucket")

print("Newly added columns:", new_columns)
display(processed[new_columns].head())
```

```
Newly added columns: ['age_group', 'balance_bucket', 'campaign_bucket', 'pda
ys_recency_bucket']
```

	age_group	balance_bucket	campaign_bucket	pdays_recency_bucket
0	51+	high	1	no_previous_contact
1	36-50	low	1	no_previous_contact
2	26-35	low	1	no_previous_contact
3	36-50	medium	1	no_previous_contact
4	26-35	low	1	no_previous_contact

2.5. Review newly engineered features against the target

For each engineered feature that exists in `processed`, show a compact summary of category counts, proportions, and subscription rate vs `y_binary`. Read-only — does not modify `processed`.

In [44]:
```
CANDIDATE_COLS = [
    "contact_is_unknown",
    "poutcome_is_unknown",
    "education_is_unknown",
    "job_is_unknown",
    "multiple_contacts_current_campaign",
    "poutcome_success",
    "poutcome_failure",
    "contacted_before_and_success",
    "contacted_before_and_failure",
    "many_contacts_and_housing_loan",
    "previous_and_many_current_contacts",
    "both_housing_and_personal_loan",
    "history_info_missing",
    "age_group",
    "balance_bucket",
    "campaign_bucket",
    "pdays_recency_bucket",
]
```

In [ ]:
```
existing_candidate_cols = [c for c in CANDIDATE_COLS if c in processed.colum

binary_check = pd.DataFrame(
    {"n_unique": [processed[c].nunique(dropna=False) for c in existing_candi
    index=existing_candidate_cols,
)

display(binary_check.sort_values("n_unique"))
```

	n_unique
contact_is_unknown	2
poutcome_is_unknown	2
multiple_contacts_current_campaign	2
poutcome_success	2
poutcome_failure	2
contacted_before_and_success	2
contacted_before_and_failure	2
many_contacts_and_housing_loan	2
previous_and_many_current_contacts	2
both_housing_and_personal_loan	2
history_info_missing	2
campaign_bucket	3
age_group	4
balance_bucket	4
pdays_recency_bucket	4

In [46]:
```
binary_cols = [
    "contact_is_unknown",
    "poutcome_is_unknown",
    "multiple_contacts_current_campaign",
    "poutcome_success",
    "poutcome_failure",
    "contacted_before_and_success",
    "contacted_before_and_failure",
    "many_contacts_and_housing_loan",
    "previous_and_many_current_contacts",
    "both_housing_and_personal_loan",
    "history_info_missing",
]

cat_cols = [
    "campaign_bucket",
    "age_group",
    "balance_bucket",
    "pdays_recency_bucket",
]
```

In [ ]:
```
# Review binary engineered features by comparing counts and subscription rat
rows = []

for col in binary_cols:
    grp = processed.groupby(col)["y_binary"].agg(
        count="count", subscription_rate="mean"
```

```
    )
    grp.index = grp.index.map(lambda v: f"{col}={v}")
    rows.append(grp)

display(pd.concat(rows).round(3))
```

	count	subscription_rate
contact_is_unknown=0	32191	0.148
contact_is_unknown=1	13020	0.041
poutcome_is_unknown=0	8252	0.231
poutcome_is_unknown=1	36959	0.092
multiple_contacts_current_campaign=0	17544	0.146
multiple_contacts_current_campaign=1	27667	0.099
poutcome_success=0	43700	0.099
poutcome_success=1	1511	0.647
poutcome_failure=0	40310	0.116
poutcome_failure=1	4901	0.126
contacted_before_and_success=0	43700	0.099
contacted_before_and_success=1	1511	0.647
contacted_before_and_failure=0	40310	0.116
contacted_before_and_failure=1	4901	0.126
many_contacts_and_housing_loan=0	30199	0.141
many_contacts_and_housing_loan=1	15012	0.068
previous_and_many_current_contacts=0	40969	0.108
previous_and_many_current_contacts=1	4242	0.204
both_housing_and_personal_loan=0	40844	0.123
both_housing_and_personal_loan=1	4367	0.061
history_info_missing=0	32261	0.148
history_info_missing=1	12950	0.040

2.6. Define the final engineered feature set

Create a compact keep/drop split for engineered features based on the earlier review step.

In [48]:
```
candidate_keep = [
    "poutcome_success",
    "contact_is_unknown",
```

```
    "poutcome_is_unknown",
    "multiple_contacts_current_campaign",
    "previous_and_many_current_contacts",
    "many_contacts_and_housing_loan",
    "both_housing_and_personal_loan",
    "age_group",
    "balance_bucket",
    "campaign_bucket",
    "pdays_recency_bucket",
]

candidate_drop = [
    "contacted_before_and_success",
    "contacted_before_and_failure",
    "history_info_missing",
    "poutcome_failure",
]

keep_features = [c for c in candidate_keep if c in processed.columns]
drop_features = [c for c in candidate_drop if c in processed.columns]

print("Keep engineered features:")
print(keep_features)

print("\nDrop engineered features:")
print(drop_features)

print(f"\nNumber of kept engineered features: {len(keep_features)}")
print(f"Number of dropped engineered features: {len(drop_features)}")
```

```
Keep engineered features:
['poutcome_success', 'contact_is_unknown', 'poutcome_is_unknown', 'multiple_
contacts_current_campaign', 'previous_and_many_current_contacts', 'many_cont
acts_and_housing_loan', 'both_housing_and_personal_loan', 'age_group', 'bala
nce_bucket', 'campaign_bucket', 'pdays_recency_bucket']

Drop engineered features:
['contacted_before_and_success', 'contacted_before_and_failure', 'history_in
fo_missing', 'poutcome_failure']

Number of kept engineered features: 11
Number of dropped engineered features: 4
```

2.7. Create the final modeling table

Build a cleaned final table for modeling by removing the dropped engineered features and separating features from the target.

In [49]:
```
processed_final = processed.drop(columns=drop_features).copy()

target_col = "y_binary"
feature_cols = [c for c in processed_final.columns if c not in ["y", target_

X_model = processed_final[feature_cols].copy()
y_model = processed_final[target_col].copy()
```

```
print("processed_final shape:", processed_final.shape)
print("X_model shape:", X_model.shape)
print("y_model shape:", y_model.shape)
display(X_model.head())
```

```
processed_final shape: (45211, 37)
X_model shape: (45211, 35)
y_model shape: (45211,)
```

	age	job	marital	education	default	balance	housing	loan	contact	day
0	58	management	married	tertiary	no	2143	yes	no	unknown	5
1	44	technician	single	secondary	no	29	yes	no	unknown	5
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5
4	33	unknown	single	unknown	no	1	no	no	unknown	5

5 rows × 35 columns

In [50]:
```
processed_final.columns
```

Out[50]:
```
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housin
g',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'y', 'y_binary', 'pdays_was_contacted',
       'pdays_since_prev', 'month_ord', 'education_ord', 'default_binary',
       'housing_binary', 'loan_binary', 'balance_signed_log',
       'multiple_contacts_current_campaign', 'poutcome_success',
       'contact_is_unknown', 'poutcome_is_unknown',
       'many_contacts_and_housing_loan', 'previous_and_many_current_contact
s',
       'both_housing_and_personal_loan', 'age_group', 'balance_bucket',
       'campaign_bucket', 'pdays_recency_bucket'],
      dtype='object')
```

2.8. Final feature list for pre-contact modeling

In [ ]:
```
selected_features = [
    "job",
    "marital",
    "day",
    "previous",
    "age_group",
    "balance_bucket",
    "campaign_bucket",
    "pdays_was_contacted",
    "pdays_recency_bucket",
    "month_ord",
    "education_ord",
    "default_binary",
    "housing_binary",
```

```python
    "loan_binary",
    "poutcome_success",
    "contact_is_unknown",
    "poutcome_is_unknown",
    "multiple_contacts_current_campaign",
    "previous_and_many_current_contacts",
    "many_contacts_and_housing_loan",
    "both_housing_and_personal_loan",
]

X_model = processed_final[selected_features].copy()
y_model = processed_final["y_binary"].copy()

excluded_cols = processed_final.columns.difference(
    selected_features + ["y", "y_binary"]
).tolist()

print("Selected features:")
print(selected_features)

print(f"\nNumber of selected features: {len(selected_features)}")

print("\nExcluded columns:")
print(excluded_cols)

print("\nX_model shape:", X_model.shape)
print("y_model shape:", y_model.shape)

display(X_model.head())
```

```
Selected features:
['job', 'marital', 'day', 'previous', 'age_group', 'balance_bucket', 'campai
gn_bucket', 'pdays_was_contacted', 'pdays_recency_bucket', 'month_ord', 'edu
cation_ord', 'default_binary', 'housing_binary', 'loan_binary', 'poutcome_su
ccess', 'contact_is_unknown', 'poutcome_is_unknown', 'multiple_contacts_curr
ent_campaign', 'previous_and_many_current_contacts', 'many_contacts_and_hous
ing_loan', 'both_housing_and_personal_loan']

Number of selected features: 21

Excluded columns:
['age', 'balance', 'balance_signed_log', 'campaign', 'contact', 'default',
'duration', 'education', 'housing', 'loan', 'month', 'pdays', 'pdays_since_p
rev', 'poutcome']

X_model shape: (45211, 21)
y_model shape: (45211,)
```

	job	marital	day	previous	age_group	balance_bucket	campaign_bucket
0	management	married	5	0	51+	high	1
1	technician	single	5	0	36-50	low	1
2	entrepreneur	married	5	0	26-35	low	1
3	blue-collar	married	5	0	36-50	medium	1
4	unknown	single	5	0	26-35	low	1

In [52]:
```
X_model.columns
```

Out[52]:
```
Index(['job', 'marital', 'day', 'previous', 'age_group', 'balance_bucket',
       'campaign_bucket', 'pdays_was_contacted', 'pdays_recency_bucket',
       'month_ord', 'education_ord', 'default_binary', 'housing_binary',
       'loan_binary', 'poutcome_success', 'contact_is_unknown',
       'poutcome_is_unknown', 'multiple_contacts_current_campaign',
       'previous_and_many_current_contacts', 'many_contacts_and_housing_loa
n',
       'both_housing_and_personal_loan'],
      dtype='object')
```

2.9. save processed table

In [ ]:
```
x_out_path = Path("data/bank-full-X-model.parquet")
y_out_path = Path("data/bank-full-y-model.parquet")

X_model.to_parquet(x_out_path, index=False)
y_model.to_frame(name="y_binary").to_parquet(y_out_path, index=False)

print(f"Saved X_model: {x_out_path.resolve()}")
print(f"Saved y_model: {y_out_path.resolve()}")
```

```
Saved X_model: /Users/far/data/bank-full-X-model.parquet
Saved y_model: /Users/far/data/bank-full-y-model.parquet
```

3. Data Preprocessing

3.1. define preprocessing column groups

In [ ]:
```
categorical_cols = [
    "job",
    "marital",
    "age_group",
    "balance_bucket",
    "campaign_bucket",
    "pdays_recency_bucket",
]

numeric_cols = [
    "day",
```

```
    "previous",
    "pdays_was_contacted",
    "month_ord",
    "education_ord",
    "default_binary",
    "housing_binary",
    "loan_binary",
    "poutcome_success",
    "contact_is_unknown",
    "poutcome_is_unknown",
    "multiple_contacts_current_campaign",
    "previous_and_many_current_contacts",
    "many_contacts_and_housing_loan",
    "both_housing_and_personal_loan",
]

print("Categorical columns:")
print(categorical_cols)

print("\nNumeric / binary columns:")
print(numeric_cols)

print("\nTotal selected columns:", len(categorical_cols) + len(numeric_cols)
print("X_model columns:", X_model.shape[1])
```

```
Categorical columns:
['job', 'marital', 'age_group', 'balance_bucket', 'campaign_bucket', 'pdays_
recency_bucket']

Numeric / binary columns:
['day', 'previous', 'pdays_was_contacted', 'month_ord', 'education_ord', 'de
fault_binary', 'housing_binary', 'loan_binary', 'poutcome_success', 'contact
_is_unknown', 'poutcome_is_unknown', 'multiple_contacts_current_campaign',
'previous_and_many_current_contacts', 'many_contacts_and_housing_loan', 'bot
h_housing_and_personal_loan']

Total selected columns: 21
X_model columns: 21
```

3.2. Define the preprocessing transformer

Create a preprocessing pipeline that one-hot encodes categorical features and scales numeric/binary features before modeling.

In [61]:
```
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ("num", StandardScaler(), numeric_cols),
    ]
)

print(preprocessor)
```

```
ColumnTransformer(transformers=[('cat', OneHotEncoder(handle_unknown='ignor
e'),
                                 ['job', 'marital', 'age_group',
                                  'balance_bucket', 'campaign_bucket',
                                  'pdays_recency_bucket']),
                                ('num', StandardScaler(),
                                 ['day', 'previous', 'pdays_was_contacted',
                                  'month_ord', 'education_ord',
                                  'default_binary', 'housing_binary',
                                  'loan_binary', 'poutcome_success',
                                  'contact_is_unknown', 'poutcome_is_unknow
n',
                                  'multiple_contacts_current_campaign',
                                  'previous_and_many_current_contacts',
                                  'many_contacts_and_housing_loan',
                                  'both_housing_and_personal_loan'])])
```

3.3 Split the data and apply preprocessing

Split the data into training and validation sets, then fit the preprocessing pipeline on the training set only and transform both sets.

In [ ]:
```
X_train, X_val, y_train, y_val = train_test_split(
    X_model,
    y_model,
    test_size=0.2,
    random_state=42,
    stratify=y_model,
)

X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed = preprocessor.transform(X_val)

print("X_train shape:", X_train.shape)
print("X_val shape:", X_val.shape)
print("y_train shape:", y_train.shape)
print("y_val shape:", y_val.shape)

print("\nX_train_processed shape:", X_train_processed.shape)
print("X_val_processed shape:", X_val_processed.shape)
```

```
X_train shape: (36168, 21)
X_val shape: (9043, 21)
y_train shape: (36168,)
y_val shape: (9043,)

X_train_processed shape: (36168, 45)
X_val_processed shape: (9043, 45)
```

3.4. Inspect the transformed feature names

Retrieve the final feature names after one-hot encoding and scaling to see the exact modeling input.

In [64]:
```
feature_names = preprocessor.get_feature_names_out()

print("Number of transformed features:", len(feature_names))
print("\nFirst 30 feature names:")
print(feature_names[:30])
```

```
Number of transformed features: 45

First 30 feature names:
['cat__job_admin.' 'cat__job_blue-collar' 'cat__job_entrepreneur'
 'cat__job_housemaid' 'cat__job_management' 'cat__job_retired'
 'cat__job_self-employed' 'cat__job_services' 'cat__job_student'
 'cat__job_technician' 'cat__job_unemployed' 'cat__job_unknown'
 'cat__marital_divorced' 'cat__marital_married' 'cat__marital_single'
 'cat__age_group_18-25' 'cat__age_group_26-35' 'cat__age_group_36-50'
 'cat__age_group_51+' 'cat__balance_bucket_high' 'cat__balance_bucket_low'
 'cat__balance_bucket_medium' 'cat__balance_bucket_negative'
 'cat__campaign_bucket_1' 'cat__campaign_bucket_2-3'
 'cat__campaign_bucket_4+' 'cat__pdays_recency_bucket_medium'
 'cat__pdays_recency_bucket_no_previous_contact'
 'cat__pdays_recency_bucket_old' 'cat__pdays_recency_bucket_recent']
```

3.5. Convert the transformed arrays to data frames

Convert the preprocessed training and validation sets into DataFrames with feature names for easier inspection.

In [ ]:
```
X_train_processed_df = pd.DataFrame(
    (
        X_train_processed.toarray()
        if hasattr(X_train_processed, "toarray")
        else X_train_processed
    ),
    columns=feature_names,
    index=X_train.index,
)

X_val_processed_df = pd.DataFrame(
    (
        X_val_processed.toarray()
        if hasattr(X_val_processed, "toarray")
        else X_val_processed
    ),
    columns=feature_names,
    index=X_val.index,
)

print("X_train_processed_df shape:", X_train_processed_df.shape)
print("X_val_processed_df shape:", X_val_processed_df.shape)

display(X_train_processed_df.head())
```

```
X_train_processed_df shape: (36168, 45)
X_val_processed_df shape: (9043, 45)
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid
24001	0.0	0.0	0.0	0.0
43409	0.0	0.0	0.0	0.0
20669	0.0	0.0	0.0	0.0
18810	0.0	0.0	0.0	0.0
23130	0.0	0.0	0.0	0.0

5 rows × 45 columns

3.6. Check the transformed data

Run a quick sanity check on the preprocessed training and validation sets to confirm there are no missing or constant features.

In [ ]:
```
print(
    "Missing values in X_train_processed_df:", X_train_processed_df.isna().s
)
print("Missing values in X_val_processed_df:", X_val_processed_df.isna().sum

constant_cols = [
    c for c in X_train_processed_df.columns if X_train_processed_df[c].nunic
]

print("Number of constant columns:", len(constant_cols))
print("Constant columns:")
print(constant_cols)
```

```
Missing values in X_train_processed_df: 0
Missing values in X_val_processed_df: 0
Number of constant columns: 0
Constant columns:
[]
```